Gestión de transacciones

El término transacción hace referencia a un conjunto de operaciones que forman una única unidad lógica, es decir, que deben ser tratadas de forma atómica o en otras palabras como una única operación.

Resulta importante que, o bien se ejecuten completamente todas las acciones de una transacción, o bien, en caso de fallo, se deshagan los efectos parciales de la transacción. Esta propiedad se denomina **atomicidad**. Además, una vez ejecutada con éxito una transacción, sus efectos deben persistir en la base de datos: un fallo en el sistema no debe tener como consecuencia que la base de datos se olvide de una transacción que haya completado con éxito. Esta propiedad se denomina **durabilidad.**

En los sistemas de bases de datos en los que se ejecutan de manera concurrente varias transacciones, si no se controlan las actualizaciones de los datos compartidos existe la posibilidad de que las transacciones vean estados intermedios inconsistentes creados por las actualizaciones de otras transacciones. Esta situación puede dar lugar a actualizaciones erróneas de los datos almacenados en la base de datos. Por tanto, los sistemas de bases de datos deben proporcionar los mecanismos para aislar las transacciones de otras transacciones que se ejecuten de manera concurrente. Esta propiedad se denomina **aislamiento**.

Conceptos

Como se mencionó anteriormente, una transacción es un conjunto de instrucciones ejecutadas como una unidad, que accede y posiblemente actualiza varios datos.

Una transacción se inicia por la ejecución de un programa de usuario escrito en un lenguaje de manipulación de datos de alto nivel o en un lenguaje de programación (por ejemplo SQL, COBOL, C, C++ o Java), y está delimitado por instrucciones (o llamadas a función) de la forma ***inicio_transacción*** y ***fin_transacción***.

Con el objetivo de asegurar la integridad de los datos se necesita que el sistema de base de datos mantenga las siguientes propiedades:

- **Atomicidad**: o bien todas las operaciones de la transacción se realizan adecuadamente en la base de datos o ninguna de ellas.
- **Consistencia**: la ejecución aislada de la transacción (es decir, sin otra transacción que se ejecute concurrentemente) conserva la consistencia de la base de datos.

- **Aislamiento**: aunque se ejecuten varias transacciones concurrentemente, el sistema garantiza que para cada par de transacciones Ti y Tj, se cumple que para los efectos de Ti, o bien Tj ha terminado su ejecución antes de que comience Ti , o bien que Tj ha comenzado su ejecución después de que Ti termine. De este modo, cada transacción ignora al resto de las transacciones que se ejecuten concurrentemente en el sistema.
- **Durabilidad**: Tras la finalización con éxito de una transacción, los cambios realizados en la base de datos permanecen, incluso si hay fallos en el sistema.

Estas propiedades reciben el nombre de propiedades **ACID**; el acrónimo se obtiene de la primera letra de cada una de las cuatro propiedades en inglés (Atomicity, Consistency, Isolation and Durability, respectivamente).

Veamos un ejemplo para comprender mejor cómo impactan cada una de estas propiedades en la operatoria de una base de datos. Consideremos un sistema bancario (simplificado) donde se requiere actualizar los saldos de las cuentas de los clientes. Vamos a considerar, para simplificar el ejemplo, que los datos de la base de datos residen únicamente en disco y que las escrituras se realizan inmediatamente a disco.

Sea Ti una transacción para transferir $50 de la cuenta A a la cuenta B. Se puede definir dicha transacción como

```
Ti:
   leer (A);
   A := A – 50;
   escribir (A);
   leer (B);
   B := B + 50;
   escribir (B).
```

Veamos cómo aplican cada una de las propiedades **ACID**:

Consistencia: en este caso el requisito de consistencia es que la suma de A y B no sea alterada al ejecutar la transacción. Sin el requisito de consistencia, la transacción podría crear o destruir dinero. La responsabilidad de asegurar la consistencia de una transacción es del programador de la aplicación que codifica dicha transacción. La comprobación automática de las restricciones de integridad puede facilitar esta tarea.

Atomicidad: supóngase que justo antes de ejecutar la transacción Ti los valores de las cuentas A y B son de $1.000 y de $2.000, respectivamente. Supóngase ahora que durante la ejecución de la transacción Ti se produce un fallo que impide que dicha transacción finalice con éxito su ejecución. Ejemplos de este tipo de fallos pueden ser los fallos en la alimentación, los fallos del

hardware y los errores software. Además, supóngase que el fallo tiene lugar después de ejecutarse la operación escribir (A), pero antes de ejecutarse la operación escribir (B). Es ese caso, los valores de las cuentas A y B que se ven reflejados en la base de datos son $950 y $2.000. Se han perdido $50 de la cuenta A como resultado de este fallo.

En particular se puede ver que ya no se conserva la suma A + B. Así, como resultado del fallo, el estado del sistema deja de reflejar el estado real del mundo que se supone que modela la base de datos. Un estado así se denomina estado inconsistente.

Sin embargo, un sistema puede en algún momento alcanzar un estado inconsistente, incluso si la transacción Ti se ejecuta por completo, existe un punto intermedio en el que el valor de la cuenta A es de $950 y el de la cuenta B es de $2.000, lo cual constituye claramente un estado inconsistente. Este estado, sin embargo, se sustituye eventualmente por otro estado consistente en el que el valor de la cuenta A es de $950 y el de la cuenta B es de $2.050. De este modo, si la transacción no empieza nunca o se garantiza que se complete, un estado inconsistente así no será visible excepto durante la ejecución de la transacción. Ésta es la razón de que aparezca el requisito de atomicidad. Si se proporciona la propiedad de atomicidad, o todas las acciones de la transacción se ven reflejadas en la base de datos, o ninguna de ellas.

La idea básica que hay detrás de asegurar la atomicidad es la siguiente: el sistema de base de datos mantiene los valores antiguos (en disco) de aquellos datos sobre los que una transacción realiza una escritura y, si la transacción no completa su ejecución, los valores antiguos se recuperan para que parezca que la transacción no se ha ejecutado.

La responsabilidad de asegurar la atomicidad es del sistema de base de datos; en concreto, lo maneja un componente llamado componente de gestión de transacciones

Durabilidad: la propiedad de durabilidad asegura que, una vez que se completa con éxito una transacción, persisten todas las modificaciones realizadas en la base de datos, incluso si hay un fallo en el sistema después de completarse la ejecución de dicha transacción.

A partir de ahora se asume que un fallo en la computadora del sistema produce una pérdida de datos de la memoria principal, pero los datos almacenados en disco nunca se pierden. Se puede garantizar la durabilidad si se asegura que:

1. Las modificaciones realizadas por la transacción se guardan en disco antes de que finalice la transacción.
2. La información de las modificaciones realizadas por la transacción guardada en disco es suficiente para permitir a la base de datos reconstruir dichas modificaciones cuando el sistema se reinicie después del fallo

La responsabilidad de asegurar la durabilidad es de un componente del sistema de base de datos llamado componente de gestión de recuperaciones o gestor de integridad.

Aislamiento: incluso si se aseguran las propiedades de consistencia y de atomicidad para cada transacción, si varias transacciones se ejecutan concurrentemente, se pueden entrelazar sus operaciones de un modo no deseado, produciendo un estado inconsistente.

Por ejemplo, como se ha visto antes, la base de datos es inconsistente temporalmente durante la ejecución de la transacción para transferir fondos de la cuenta A a la cuenta B, con el total deducido escrito ya en A y el total incrementado todavía sin escribir en B. Si una segunda transacción que se ejecuta concurrente lee A y B en este punto intermedio y calcula A + B, observará un valor inconsistente. Además, si esta segunda transacción realiza después modificaciones en A y B basándose en los valores leídos, la base de datos puede permanecer en un estado inconsistente aunque ambas transacciones terminen.

Una solución para el problema de ejecutar transacciones concurrentemente es ejecutarlas secuencialmente, es decir, una tras otra. Sin embargo, la ejecución concurrente de transacciones produce notables beneficios en el rendimiento. Se han desarrollado otras soluciones que permiten la ejecución concurrente de varias transacciones. La propiedad de aislamiento asegura que el resultado obtenido al ejecutar concurrentemente las transacciones es un estado del sistema equivalente a uno obtenido al ejecutar una tras otra en algún orden.

La responsabilidad de asegurar la propiedad de aislamiento es de un componente del sistema de base de datos llamado componente de control de concurrencia.

Control de la concurrencia

Cuando se ejecutan concurrentemente varias transacciones, la base de datos puede dejar de ser consistente. Es necesario que el sistema controle la interacción entre las transacciones concurrentes, y dicho control se lleva a cabo mediante uno de los muchos mecanismos llamados esquemas de control de concurrencia.

Se pueden usar varios esquemas de control de concurrencia para asegurar la secuencialidad. Todos estos esquemas o bien retrasan (bloquean) una operación o bien abortan la transacción que ha realizado la operación. Los más comunes son los protocolos de bloqueo, los esquemas de ordenación por marcas temporales, las técnicas de validación y los esquemas multiversión.

Esquema de control de concurrencia multiversión (MVCC)

En los esquemas de control de concurrencia multiversión, donde Q es un dato, cada operación escribir(Q) crea una nueva versión de Q. Cuando se realiza una operación leer(Q) el gestor de control de concurrencia elige una de las versiones de Q que se va a leer.

El esquema de control de concurrencia debe asegurar que la elección de la versión que se va a leer se haga de tal manera que asegure la secuencialidad. Así mismo es crucial, por motivos de rendimiento, que una transacción sea capaz de determinar rápida y fácilmente la versión del elemento de datos que se va a leer.

Ordenación por marcas temporales multiversión

La técnica más frecuentemente utilizada en los esquemas multiversión es la de las marcas temporales. A cada transacción Ti del sistema se le asocia una única marca temporal estática que se denota MT(Ti). El sistema de bases de datos asigna dicha marca temporal antes de que la transacción comience su ejecución

A cada elemento de datos Q se el asocia una secuencia de versiones <Q 1, Q 2, ... , Qm>. Cada versión Qk contiene tres campos:

- contenido es el valor de la versión Qk.
- marca_temporal-E(Qk) es la marca temporal de la transacción que haya creado la versión Qk.
- marca_temporal-L(Qk) es la mayor marca temporal de todas las transacciones que hayan leído con éxito la versión Qk.

Una transacción —por ejemplo, Ti— crea una nueva versión Qk del elemento de datos Q realizando la operación escribir(Q). El campo contenido de la versión tiene el valor que ha escrito Ti. El sistema inicializa la marca_temporal-E y marca_temporal-L con MT(Ti). El valor marca_temporal-L se actualiza cada vez que una transacción Tj lee el contenido de Qk y marca_temporal-L(Qk) < MT(Tj).

El esquema de marcas temporales multiversión que se muestra a continuación asegura la secuencialidad. El esquema opera como sigue. Supóngase que la transacción T i realiza una operación leer(Q) o escribir(Q). Sea Qk la versión de Q cuya marca temporal de escritura es la mayor marca temporal menor o igual que MT(Ti).

1. Si la transacción T i ejecuta leer (Q), entonces el valor que se devuelve es el contenido de la versión Q k.

2. Si la transacción Ti ejecuta escribir(Q) y si MT(Ti) < marca_temporal-L(Qk), entonces la transacción Ti se retrocede. Caso contrario, si MT(Ti) = marca_temporal-E(Qk) se sobrescribe el contenido de Qk, y en otro caso se crea una nueva versión de Q.

La justificación de la regla 1 es que una transacción lea la versión más reciente/actual que viene antes de ella en el tiempo.

La segunda regla fuerza a que se aborte una transacción que realice una escritura «demasiado tarde». Con más precisión, si Ti intenta escribir una versión que alguna otra transacción haya leído, entonces no se puede permitir que dicha escritura tenga éxito.

El esquema de ordenación por marcas temporales multiversión tiene la propiedad deseable de que nunca falla una petición de lectura y nunca tiene que esperar. En sistemas de bases de datos típicos, en los cuales la lectura es una operación mucho más frecuente que la escritura, esta ventaja puede tener un mayor significado práctico.

Sin embargo, tiene dos propiedades no deseables. En primer lugar, la lectura de un elemento de datos necesita también la actualización del campo marca_temporal-L, lo que tiene como resultado dos accesos potenciales al disco en lugar de uno. En segundo lugar, los conflictos entre las transacciones se resuelven por medio de retrocesos en lugar de esperas. Esta alternativa puede ser costosa.

Bloqueo de dos fases multiversión

Este protocolo distingue entre transacciones de sólo lectura y transacciones de actualización. Las transacciones de actualización mantienen todos los bloqueos hasta el final de la transacción. Así, se puede secuenciar según su orden de terminación. Cada elemento de datos tiene una única marca temporal. La marca temporal no es en este caso una marca temporal basada en un reloj real, sino que es un contador, que se denomina contador_mt, que se incrementa durante el procesamiento del compromiso.

A las transacciones de sólo lectura se les asigna una marca temporal leyendo el valor actual de contador_mt antes de que comiencen su ejecución; para realizar las lecturas siguen el protocolo de ordenación por marcas temporales multiversión. Así, cuando una transacción de sólo lectura Ti ejecuta leer(Q), el valor que se devuelve es el contenido de la versión con la mayor marca temporal menor que MT(Ti).

Cuando una transacción de actualización lee un elemento obtiene un bloqueo compartido sobre él y lee la versión más reciente de dicho elemento de datos. Cuando una transacción de actualización quiere escribir un elemento, obtiene primero un bloqueo exclusivo sobre el elemento, y luego crea una nueva versión del elemento de datos. La escritura se realiza sobre la

nueva versión y la marca temporal de la nueva versión tiene inicialmente el valor ∞, el cual es mayor que el de cualquier marca temporal posible.

Una vez que la transacción Ti ha completado sus acciones realiza el procesamiento del compromiso como sigue:

1. en primer lugar, Ti asigna la marca temporal de todas las versiones que ha creado al valor de contador_mt más 1;
2. después Ti incrementa contador_mt en 1. Sólo se permite que realice el procesamiento del compromiso a una transacción de actualización a la vez.

Como resultado, las transacciones que comiencen después de que Ti incremente el contador_mt se observarán los valores que Ti ha actualizado, mientras que aquellas que comiencen antes de que Ti incremente el contador_mt se observarán los valores anteriores a la actualización de Ti. En ambos casos las transacciones de sólo lectura no tienen que esperar nunca por los bloqueos.

El bloqueo de dos fases multiversión o variaciones de éste se usan en algunos sistemas de bases de datos comerciales.

Implementación en PostgreSQL

PostgreSQL mantiene la coherencia de los datos mediante el uso de un modelo de múltiples versiones (Control de concurrencia de múltiples versiones, MVCC). Esto significa que cada instrucción SQL ve una instantánea de los datos (una versión de la base de datos), independientemente del estado actual de los datos subyacentes. Esto evita que se vean datos inconsistentes producidos por transacciones concurrentes que realizan actualizaciones en las mismas filas de datos, proporcionando aislamiento de transacciones para cada sesión de base de datos. MVCC, al evitar las metodologías de bloqueo de los sistemas de base de datos tradicionales, minimiza la contención de bloqueo para permitir un rendimiento razonable en entornos multiusuario.

La principal ventaja de usar el modelo MVCC de control de concurrencia en lugar de bloquear es que en MVCC los bloqueos adquiridos para consultar (leer) datos no entran en conflicto con los bloqueos adquiridos para escribir datos, por lo que la lectura nunca bloquea la escritura y la escritura nunca bloquea la lectura. PostgreSQL mantiene esta garantía incluso cuando proporciona el nivel más estricto de aislamiento de transacciones mediante el uso de un innovador nivel de aislamiento de instantáneas serializables (SSI).

Las funciones de bloqueo a nivel de tabla y de fila también están disponibles en PostgreSQL para aplicaciones que generalmente no necesitan un aislamiento total de transacciones y prefieren administrar explícitamente puntos particulares de conflicto.

Niveles de aislamiento

El estándar SQL define cuatro niveles de aislamiento de transacciones:

- Read uncommitted
- Read committed
- Repeatable read
- Serializable

El más estricto es ***Serializable***, el mismo establece que cualquier ejecución concurrente de un conjunto de transacciones serializables está garantizado para producir el mismo efecto que ejecutarlas una a la vez en algún orden.

En el estudio de las transacciones concurrentes, surgen algunos fenómenos que deben ser tenidos en cuenta al momento de diseñar una aplicación. Estos fenómenos son problemas generados por la posible concurrencia de las transacciones. Tengamos en cuenta que una base de datos puede ser visto como un un recurso compartido por varios procesos ya sea que se ejecutan en la misma máquina o en máquinas distribuídas a través de la red. Cabe destacar que estos fenómenos no se producen cuando se utiliza el nivel de aislamiento ***Serializable***. Los eventos son:

- **Lectura no confirmada o sucia (dirty read):** una transacción lee datos escritos por otra transacción concurrente que no confirmó los datos, es decir, la transacción está trabajando con datos temporales.
  Por ejemplo:

Transacción A	Transacción B
`BEGIN TRANSACCTION` `  update products set uprice = uprice * 2;`	

...	`SET TRANSACTION ISOLATION LEVEL READ UNCOMMITED;`
	
	`DECLARE vuprice float;`
	
	`select uprice into vuprice from products where product_id = 24;`

`rollback transaction;`	...
	`insert into fac_detalle (...., uprice, ….) values (..., vuprice, ….)`

En este ejemplo, la transacción B utiliza un precio unitario erróneo ya que la transacción A deshizo los cambios.

- **Análisis inconsistente o Lectura no repetible (non repeatable read):** una transacción vuelve a leer los datos que ha leído anteriormente y encuentra que los datos han sido modificados por otra transacción (que se confirmó desde la lectura inicial).

  Por ejemplo: si se ejecuta un proceso de larga duración, digamos un reporte mensual de facturación, puede ocurrir que durante la ejecución del reporte, se actualicen algunas ventas (otras transacciones se ejecuten) lo que llevará a generar datos erróneos por ejemplo que el total no coincida con el detalle.

Transacción A (generación reporte)	Transacción B (pueden ser varias)

`select sum((pu * cant) * (1-descuento))`
`into total`
`from fac_detalles;`	

	`update fac_detalles set descuento`
	`= 0.4`
	`where prod_id = 10;`

`select fac_id, pu, cant, descuento` `from fac_destalles;`	

  En este caso, el total será diferente del detalle.

- **Lectura fantasma (phantom read):** similar a la situación anterior, la lectura fantasma es una anomalía en la cual una transacción vuelve a ejecutar una consulta que devuelve un conjunto de filas que satisfacen una condición de búsqueda y encuentra que el conjunto de filas que satisfacen la condición ha cambiado debido a otra transacción confirmada recientemente.
  Por ejemplo: si se ejecuta un proceso de larga duración, digamos un reporte mensual de facturación, puede ocurrir que durante la ejecución del reporte, se realicen otras ventas (otras transacciones se ejecuten) lo que llevará a generar datos erróneos por ejemplo que el total no coincida con el detalle.

Transacción A (generación reporte)	Transacción B (pueden ser varias)

`select sum((pu * cant) * (1-descuento))`
`into total`
`from fac_detalles;`	

	`insert into fac_detalles (.....) values (..... );`
`select fac_id, pu, cant, descuento from fac_destalles;`	

- **Anomalía de serialización (serialization anomaly):** se puede producir una situación de modificaciones perdidas cada vez que 2 conexiones modifican los mismos datos una después de la otra, en otras palabras, una condición de carrera.

Transacción A (generación reporte)	Transacción B (pueden ser varias)
`Estado inicial: pu = 1`	
`select pu into vpu from products where p_id = 23;`	
	`select pu into vpu from products where p_id = 23;`
`update products set pu = vpu * 1.2 where p_id = 23;`	
	`update products set pu = vpu * 1.4 where p_id = 23;`
`select vpu * 1.2, pu from products where p_id = 23;`	`select pu from products where p_id = 23;`

`vpu | pu |`
`----------`
`1.2 | 1.4`
	` pu  |`
	`----------`
	` 1.4 |`

Este problema se puede prevenir escribiendo instrucciones atómicas, por ejemplo:

```
update products set pu = pu * 1.2 where p_id = 23;
```

En PostgreSQL se pueden utilizar cualquiera de los 4 niveles de aislamiento mencionados, aunque sólo 3 se encuentren implementados.

La siguiente tabla muestra los niveles de aislamiento y los fenómenos que pueden producirse en cada uno.

Nivel/Fenómeno	Dirty read	Nonrepeatable read	Phantom read	Serialization anomaly
Read uncommited	Permitido, pero no en Postgres	✓	✓	✓
Read commited	✗	✓	✓	✓
Repeatable read	✗	✗	Permitido, pero no en Postgres	✓
Serializable	✗	✗	✗	✗

Para establecer el nivel de aislamiento de una transacción, se utiliza el comando SET TRANSACTION (https://www.postgresql.org/docs/current/sql-set-transaction.html).

Read (un)Committed Isolation Level

Es el nivel de aislamiento predeterminado en PostgreSQL. Cuando una transacción usa este nivel de aislamiento, una consulta SELECT solo ve los datos confirmados antes de que comenzara la consulta; nunca ve datos no confirmados o cambios cometidos durante la ejecución de la consulta por transacciones concurrentes.

En efecto, una consulta SELECT ve una instantánea (snapshot) de la base de datos en el momento en que la consulta comienza a ejecutarse. Esto significa que si alguna transacción concurrente está modificando los datos, la consulta SELECT no verá la modificación hasta que los cambios mencionados se hayan confirmado. Sin embargo, sí ve los efectos de las actualizaciones anteriores ejecutadas dentro de su propia transacción, aunque aún no se hayan confirmado.

Por otra parte, operaciones como INSERT, UPDATE o DELETE su comportamiento es un poco distinto en el sentido que sólo verán los datos confirmados antes de comenzar la operación. Ahora bien, puede ocurrir que al momento de comenzar la operación, los datos determinados por la cláusula WHERE estén siendo modificados por otra transacción concurrente. En ese caso, el UPDATE o DELETE se pondrá en espera a que la transacción concurrente confirme o deshaga

los cambios. Si la transacción concurrente no aplica los cambios, el UPDATE o DELETE procede como si nada hubiese ocurrido. Si la transacción concurrente confirma los cambios, entonces el WHERE vuelve a evaluarse para definir si se debe o no seleccionar la fila en dicha operación.

```
BEGIN;
UPDATE accounts SET balance = balance + 100.00 WHERE acctnum = 12345;
UPDATE accounts SET balance = balance - 100.00 WHERE acctnum = 7534;
COMMIT;
```

Si 2 transacciones como la mostrada se ejecutan de forma concurrente, no se producen errores de consistencia por los motivos ya explicados. La segunda transacción deberá esperar a que la primera finalice para luego actualizar los datos.

Veamos otro ejemplo: supongamos que la situación actual es que existen 2 registros en la tabla *website* donde uno posee 9 hits y el otro 10.

```
BEGIN;
UPDATE website SET hits = hits + 1;
-- run from another session:  DELETE FROM website WHERE hits = 10;
COMMIT;
```

Analicemos qué ocurre en esta situación:

1. El where del DELETE pre-seleccionará 1 fila, la que posee 10 en el campo hits
2. Se ejecuta el update que actualiza el campo hits en todos los registros, llevando el 9 a 10 y el 10 a 11.
3. Cuando finaliza el UPDATE, y el DELETE obtiene el bloqueo sobre las filas, las filas preseleccionadas ya no cumplen la condición por lo que no eliminará ninguna fila.

Como resumen podemos decir que este nivel de aislamiento si bien impide lecturas no confirmadas, permite que otras conexiones actualicen datos previamente leídos (lecturas no repetibles) o inserten nuevas filas (lecturas fantasma).

Tener en cuenta que otros motores de base de datos implementan por separado los niveles aquí presentados.

Para la mayoría de las aplicaciones este nivel de aislamiento proporciona un buen balance entre bloqueos y rendimiento.

Repeatable Read Isolation Level

El nivel de aislamiento de lectura repetible sólo ve los datos confirmados antes de que comenzara la transacción; nunca ve datos no confirmados o cambios cometidos durante la ejecución de transacciones por transacciones concurrentes. (Sin embargo, la consulta ve los

efectos de las actualizaciones anteriores ejecutadas dentro de su propia transacción, aunque aún no se hayan confirmado). Esta es una garantía más sólida que la requerida por el estándar SQL para este nivel de aislamiento y evita todos los fenómenos. descrito previamente excepto por las anomalías de serialización.

Este nivel es diferente de Lectura confirmada dado que una consulta en una transacción de lectura repetible ve siempre los mismos datos sin que las actualizaciones realizadas en el resto de las transacciones los afecten.

Por lo tanto, los comandos SELECT sucesivos dentro de una sola transacción ven los mismos datos, es decir, no ven los cambios realizados por otras transacciones que confirmaron los datos después de que comenzó la transacción.

Tener en cuenta que este nivel de aislamiento en otros motores de base de datos, si bien asegura lecturas consistentes no impide la aparición de lecturas fantasmas. En Postgres sí las impide.

Serializable Isolation Level

Este nivel de aislamiento es el nivel más estricto emulando las transacciones serializables como si se ejecutaran una detrás de la otra.

Utilización de transacciones

Para utilizar transacciones dentro de nuestras consultas se utilizan los comandos

- BEGIN TRANSACTION: para comenzar un bloque de operaciones
- COMMIT: para confirmar las operaciones del bloque.
- ROLLBACK: para deshacer las operaciones del bloque.

Ejemplo I:

```
BEGIN;
UPDATE accounts SET balance = balance - 100.00 WHERE name = 'Alice';
COMMIT;
```

Ejemplo II:

```
BEGIN;
UPDATE accounts SET balance = balance - 100.00 WHERE name = 'Alice';
SELECT balance into vbalance from accounts where name = ‘Alice’;
IF vbalance < 0 THEN
    ROLLBACK;
ELSE
    COMMIT;
END IF;
```

Lectura recomendada

- https://www.postgresql.org/docs/10/tutorial-transactions.html
- https://www.postgresql.org/docs/current/mvcc.html

Bibliografía

- Gavin Powell. Beginning Database Design. Indianapolis: Wiley Publishing, Inc, 2006. ISBN-13: 978-0-7645-7490-0.
- Abraham Silberschatz; Henry F. Korth; S. Sudarshan. FUNDAMENTOS DE BASES DE DATOS. Cuarta Edición. España: McGRAW-HILL/INTERAMERICANA DE ESPAÑA, S. A. U, 2002.
- C. J. Date. Introducción A Los Sistemas De Bases De Datos. Séptima Edición. Méjico: Pearson Educación, 2001. ISBN: 968-444-419-2
- PostgreSQL Current Documentation. Disponible en: https://www.postgresql.org/docs